EvalConnect

Plateforme d'évaluation en ligne sobre & éco-conçue

Rapport Final — Partie 2

Module TI616 — Numérique Durable | EFREI Paris 2025-2026

Site déployé : https://numerique-durable.onrender.com

GitHub : https://github.com/michele453/NUMERIQUE-DURABLE

Équipe :

- SIMO KAMOU Michele
- TEMGOUA Igor
- DE SOUZA manoel
- Assirem REGAD

Section 1 — Présentation du projet

1.1 Proposition de valeur

EvalConnect est une plateforme d'évaluation en ligne sobre et éco-conçue, destinée aux enseignants du secondaire et du supérieur. Elle permet de créer des quiz, de les faire passer à des étudiants et de consulter les résultats, sans recourir à des services tiers comme Google Forms ou Moodle.

La valeur apportée repose sur trois piliers : données hébergées localement (aucune transmission vers des serveurs cloud tiers), interface sobre (pages < 200 Ko), et zéro tracking publicitaire.

1.2 Périmètre MVP

Le MVP livré comprend les fonctionnalités suivantes :

- Authentification sécurisée (inscription, connexion, gestion de session bcrypt)
- CRUD complet des utilisateurs (admin)
- Création et gestion de quiz avec questions à choix multiples
- Passage de quiz par les étudiants avec affichage du score immédiat
- Tableau de bord enseignant avec statistiques
- Export CSV des scores
- Gestion de classes (groupes d'étudiants)
- Bloc Green IT sur la page d'accueil

1.3 Utilisateurs cibles

Quatre rôles ont été définis : Visiteur (accès public), Étudiant (passer des quiz), Enseignant (créer et gérer des quiz), Administrateur (gestion complète de la plateforme).

1.4 Contraintes Green IT retenues

- Pages < 200 Ko (HTML + CSS + JS)
- ≤ 10 requêtes HTTP par page
- Aucune dépendance JavaScript lourde (pas de React, Vue, Angular)
- Typographie système (pas de Google Fonts)
- Aucun cookie de tracking, aucune donnée superflue collectée

Section 2 — Architecture et conception

2.1 Architecture générale

L'architecture d'EvalConnect est volontairement simple et sobre, structurée en trois couches :

- Front-end : HTML5 / CSS3 natif avec variables CSS, Vanilla JS minimal (<50 lignes)
- Back-end : Python 3 + Flask (micro-framework, ~30 Mo RAM en production)
- Base de données : SQLite en développement, PostgreSQL en production (Render.com)

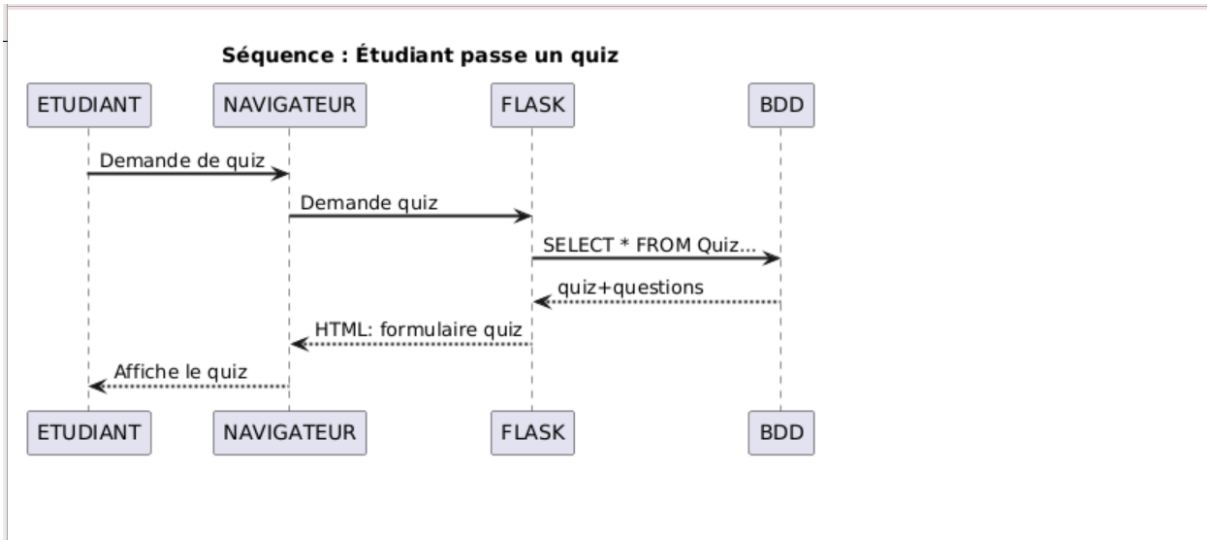
Le déploiement est assuré sur Render.com avec connexion automatique à la base PostgreSQL via variable d'environnement DATABASE_URL.

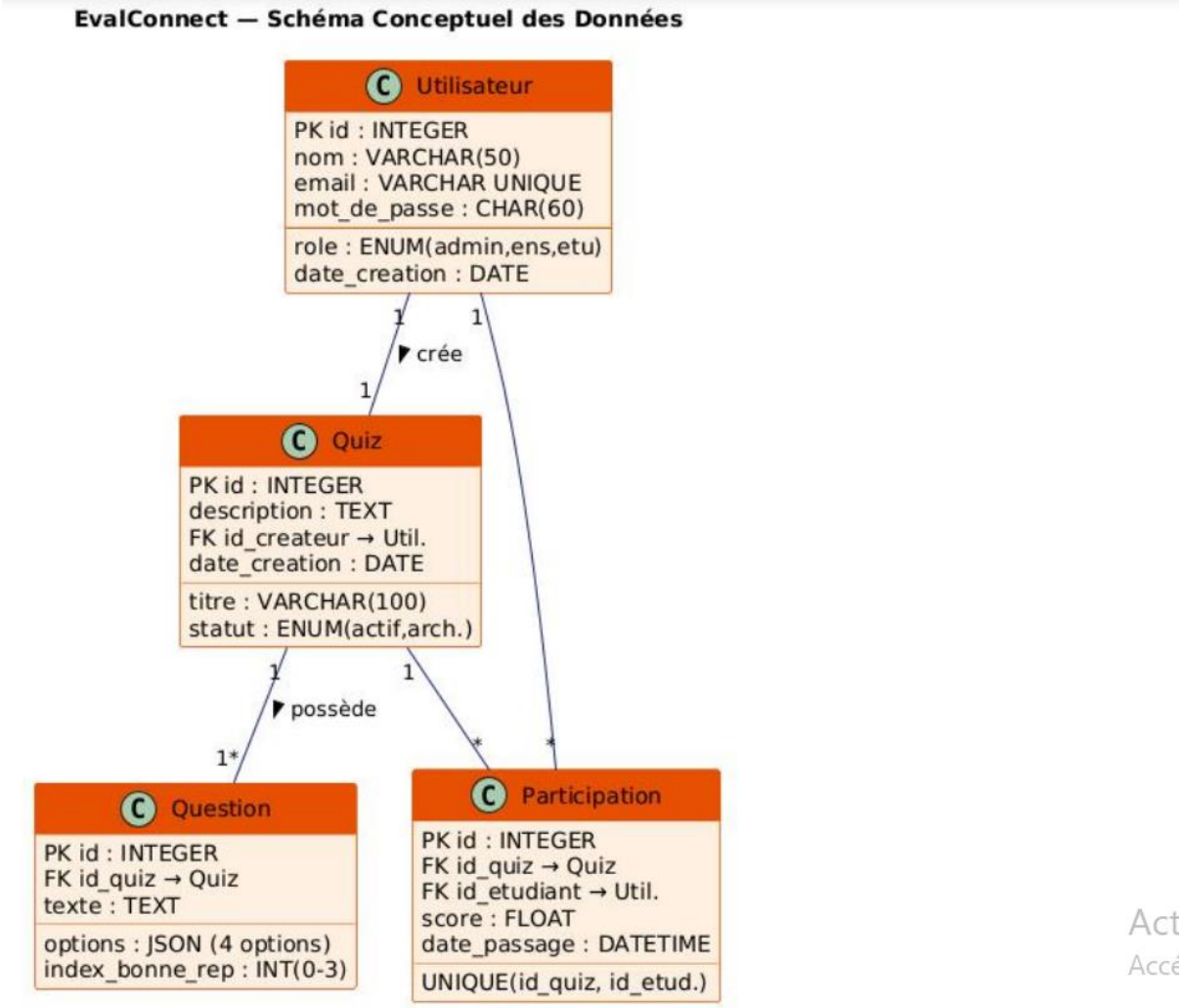
2.2 Schéma de la base de données

La base de données contient 7 tables normalisées et minimales :

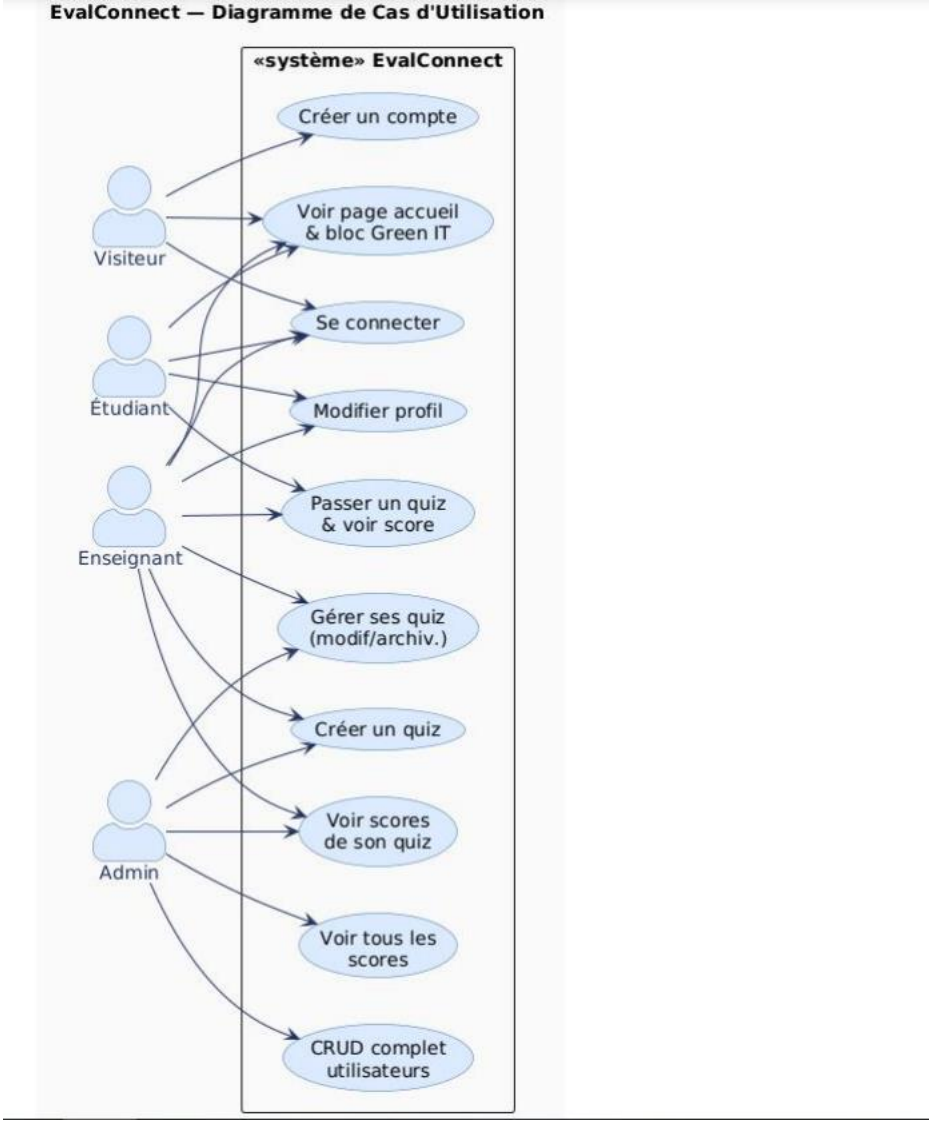
Table	Colonnes principales	Rôle
utilisateur	id, nom, prenom, email, mot_de_passe (bcrypt), role, actif	Gestion des comptes
quiz	id, titre, description, duree_minutes, statut, id_createur	Entité métier principale
question	id, id_quiz, texte, options (JSON), index_bonne_rep	Questions du quiz
participation	id, id_quiz, id_etudiant, score, date_passage	Résultats et scores
classe	id, nom, description, id_enseignant	Groupes d'étudiants
classe_etudiant	id_classe, id_etudiant	Relation many-to-many
quiz_classe	id_quiz, id_classe	Restriction d'accès par classe

2.3 Diagrammes UML


Séquence : Étudiant passe un quiz
ETUDIANT
NAVIGATEUR
FLASK
BDD
Demande de quiz
Demande quiz
SELECT * FROM Quiz...
quiz+questions
HTML: formulaire quiz
Affiche le quiz
ETUDIANT
NAVIGATEUR
FLASK
BDD


EvalConnect — Schéma Conceptuel des Données
Utilisateur
PK id : INTEGER
nom : VARCHAR(50)
email : VARCHAR UNIQUE
mot_de_passe : CHAR(60)
role : ENUM(admin,ens,etu)
date_creation : DATE
crée
Quiz
PK id : INTEGER
description : TEXT
FK id_createur → Util.
date_creation : DATE
titre : VARCHAR(100)
statut : ENUM(actif,arch.)
possède
Question
PK id : INTEGER
FK id_quiz → Quiz
texte : TEXT
options : JSON (4 options)
index_bonne_rep : INT(0-3)
Participation
PK id : INTEGER
FK id_quiz → Quiz
FK id_etudiant → Util.
score : FLOAT
date_passage : DATETIME
UNIQUE(id_quiz, id_etud.)

Act
Acc



Section 3 — Choix technologiques justifiés

Couche	Technologie choisie	Justification Green IT
Front-end	HTML5 + CSS3 natif + Vanilla JS	0 Ko de framework JS. Pages < 5 Ko. Compatible sans transpilation.
Back-end	Python 3 + Flask	~30 Mo RAM vs ~200 Mo pour Django/Spring. Routes explicites, pas d'ORM lourd.
Base de données	SQLite (dev) / PostgreSQL (prod)	SQLite = zéro serveur en dev. PostgreSQL sur Render = hébergement sobre.
Déploiement	Render.com	Conteneur Linux minimal. Plan gratuit. Pas de CDN mondial superflu.
Polices	Police système (system-ui)	Économie de 20-80 Ko par page. Zéro requête réseau externe.
Icônes	SVG inline	0 requête réseau supplémentaire. Rendu natif par le navigateur.

Authentification	bcrypt + sessions Flask	Hashage sécurisé. Pas de dépendance OAuth externe.

Section 4 — Implémentation

4.1 Fonctionnalités développées

L'ensemble du MVP a été implémenté et est fonctionnel sur le site déployé :

- Authentification : inscription avec validation serveur, connexion bcrypt, gestion de sessions, déconnexion
- CRUD Utilisateurs (admin) : création, liste paginée, modification, suppression avec confirmation, activation/désactivation
- CRUD Quiz : création avec questions dynamiques, modification, archivage, activation, suppression
- Passage de quiz : accès contrôlé, soumission, calcul de score /10, enregistrement unique
- Dashboard enseignant : statistiques, liste paginée des quiz avec filtres
- Scores : vue étudiant (ses scores) et vue enseignant (scores par quiz avec stats)
- Export CSV : téléchargement des résultats d'un quiz
- Gestion de classes : création, modification, ajout/retrait d'étudiants

4.2 Extraits de code significatifs

Connexion compatible SQLite (local) et PostgreSQL (Render) :

```
DATABASE_URL = os.environ.get('DATABASE_URL')
def query(sql, params=()):  # remplace ? par %s pour PostgreSQL
```

Hashage bcrypt et vérification anti-timing :

```
hashed = bcrypt.hashpw(pwd.encode(), bcrypt.gensalt()).decode()
pwd_ok = user and bcrypt.checkpw(pwd.encode(), user['mot_de_passe'].encode())
```

Pagination optimisée (LIMIT/OFFSET, colonnes spécifiques) :

```
SELECT id, nom, prenom, email, role FROM utilisateur LIMIT ? OFFSET ?
```

Section 5 — Analyse de l'empreinte carbone

5.1 Protocole de mesure

Les mesures ont été effectuées sur le site déployé (https://numerique-durable.onrender.com) avec 3 outils : EcoIndex, Website Carbon Calculator et PageSpeed Insights. Trois pages publiques ont été analysées : page d'accueil, page de connexion et page d'inscription.

5.2 Résultats des mesures

Indicateur	Accueil	Connexion	Inscription
Score EcoIndex	94 / 100	95 / 100	94 / 100
Note EcoIndex	A	A	A
Poids de la page	0.004 Mo (4 Ko)	0.005 Mo (5 Ko)	0.005 Mo (5 Ko)
Requêtes HTTP	2	3	3
Website Carbon	A+	A+	A+
Cleaner than	100% des sites	100% des sites	100% des sites
PageSpeed Performance	100 / 100	100 / 100	100 / 100
PageSpeed Accessibilité	98 / 100	100 / 100	100 / 100
PageSpeed Best Practices	100 / 100	100 / 100	100 / 100
PageSpeed SEO	100 / 100	100 / 100	100 / 100

Section 6 — Tests et validation

6.1 Tests fonctionnels

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Enregistré en BDD, redirection vers login	✓ OK
Créer un utilisateur (email vide)	Message d'erreur côté serveur	✓ OK
Modifier un utilisateur existant	Données mises à jour en BDD	✓ OK
Supprimer un utilisateur	Supprimé après confirmation	✓ OK
Lister les utilisateurs	Affichage paginé (20 par page)	✓ OK
Créer une entité métier (quiz)	Quiz enregistré, liste mise à jour	✓ OK
Connexion avec identifiants valides	Redirection vers espace personnel	✓ OK
Connexion avec mauvais mot de passe	Message d'erreur, pas de session créée	✓ OK
Accès page protégée sans connexion	Redirection vers la page de connexion	✓ OK

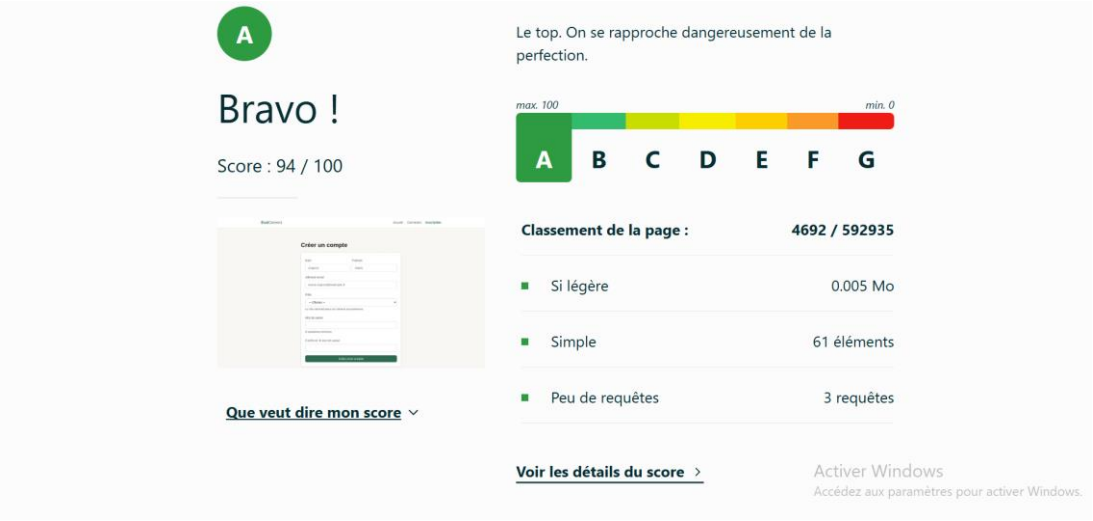
6.2 Tests de performance

PageSpeed Insights a été exécuté sur les 3 pages principales. Les scores obtenus sont :

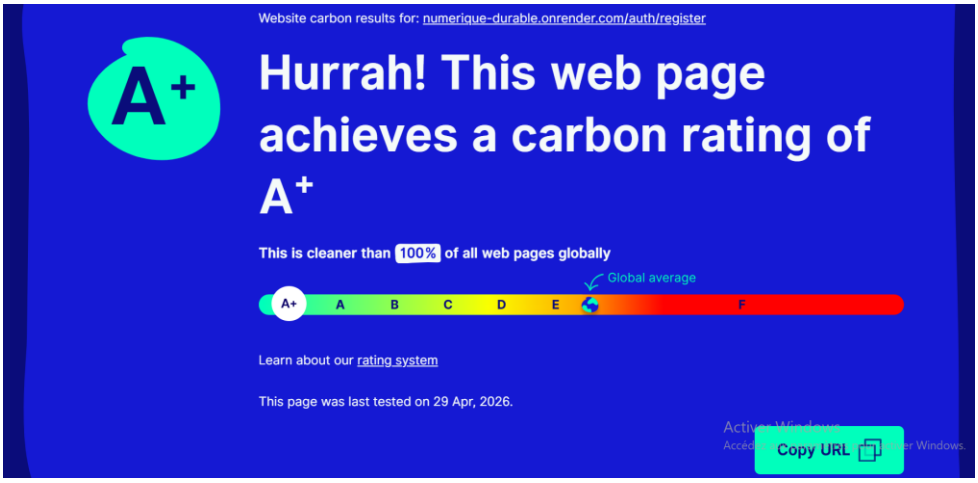
- Page d'accueil : Performance 100/100 — Accessibilité 98/100 — Best Practices 100/100 — SEO 100/100
- Page de connexion : Performance 100/100 — Accessibilité 100/100 — Best Practices 100/100 — SEO 100/100
- Page d'inscription : Performance 100/100 — Accessibilité 100/100 — Best Practices 100/100 — SEO 100/100

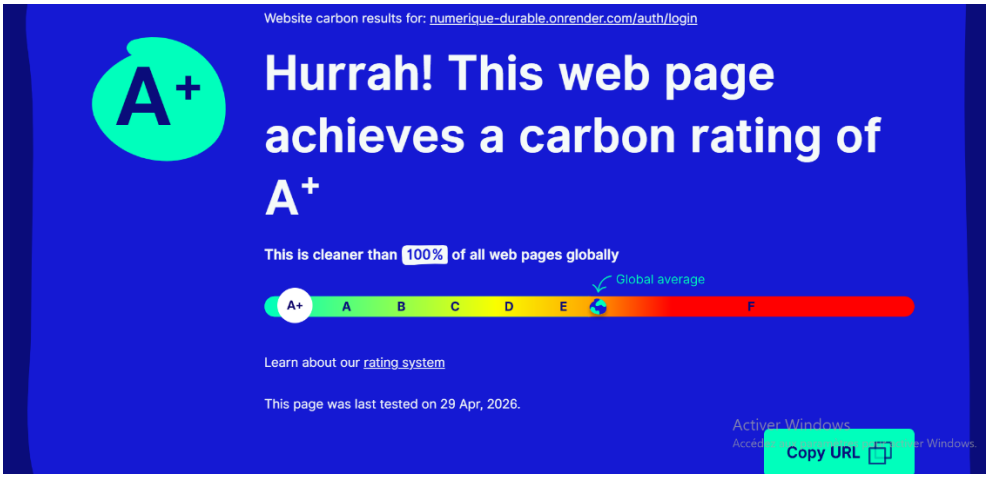
Les captures d'écran complètes des résultats pour les page register, login et le site complet:

- **EcoIndex**

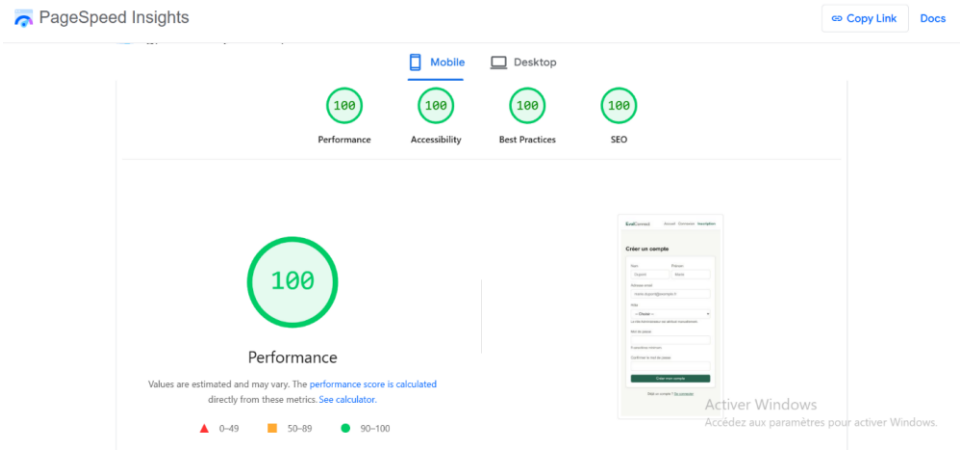


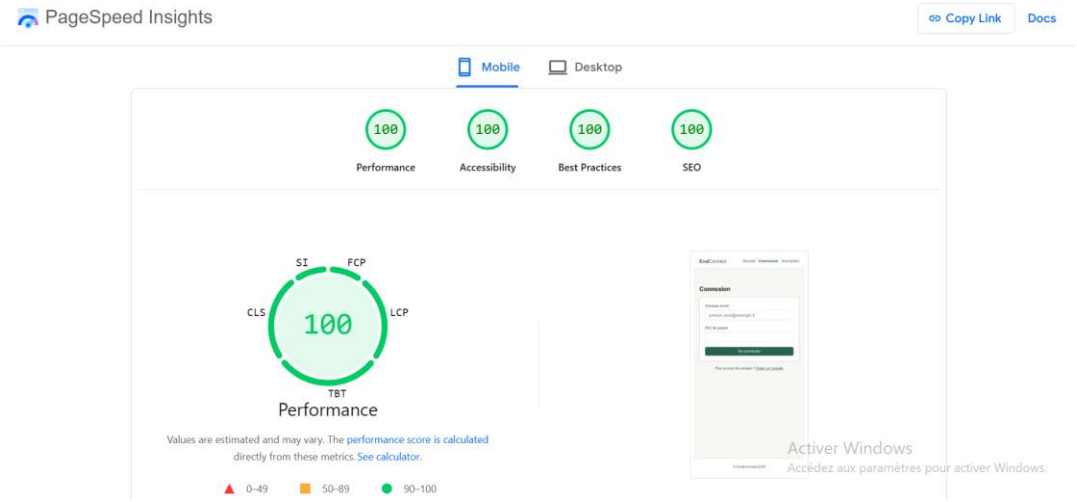
WebsiteCarbon

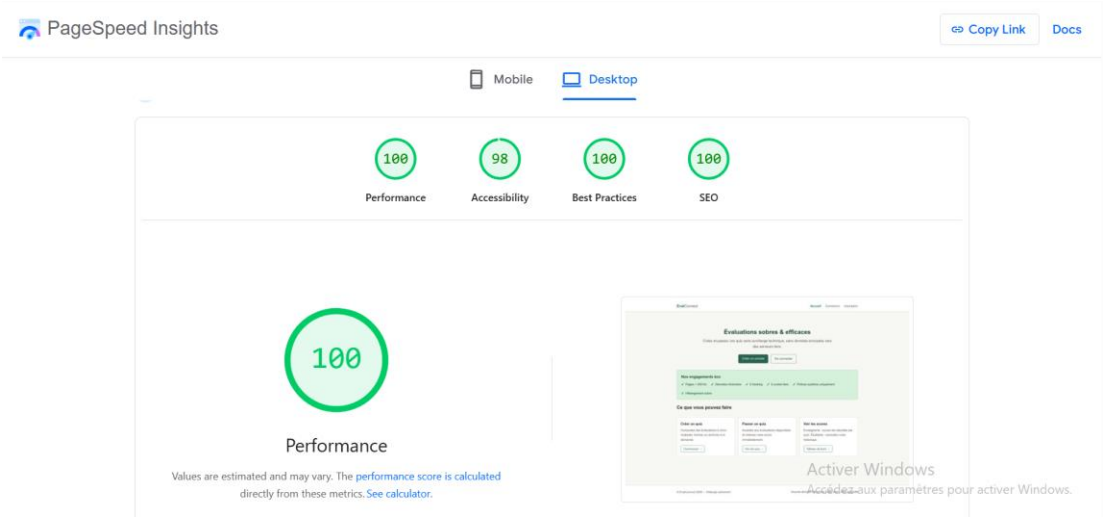




PageSpeed


PageSpeed Insights
Copy Link
Docs
Mobile
Desktop
100
Performance
100
Accessibility
100
Best Practices
100
SEO
100
Performance
Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.
0–49
50–89
90–100
Activer Windows
Accédez aux paramètres pour activer Windows.


PageSpeed Insights
Copy Link
Docs
Mobile
Desktop
100
Performance
100
Accessibility
100
Best Practices
100
SEO
SI
FCP
CLS
100
LCP
TBT
Performance
Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.
0–49
50–89
90–100
Activer Windows
Accédez aux paramètres pour activer Windows.


PageSpeed Insights
Copy Link
Docs
Mobile
Desktop
100
Performance
98
Accessibility
100
Best Practices
100
SEO
100
Performance
Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.
Activer Windows
Accédez aux paramètres pour activer Windows.

6.3 Vérifications de sécurité

Vérification	Méthode	Résultat
Mots de passe hachés	SELECT mot_de_passe FROM utilisateur	✓ Hash bcrypt
Pas de variables sensibles sur GitHub	.env dans .gitignore, .env.example fourni	✓ Vérifié
Injection SQL	Test avec '; DROP TABLE-- dans formulaire	✓ Rejeté proprement
Pages protégées sans connexion	Accès direct URL /admin/users sans session	✓ Redirection login

Section 7 — Organisation

7.1 Workflow GitHub

Le dépôt GitHub (https://github.com/michele453/NUMERIQUE-DURABLE) a été organisé avec les branches suivantes :

- main : code stable, déployé automatiquement sur Render
- feature/auth : authentification et CRUD utilisateurs
- feature/quiz : CRUD quiz et questions
- feature/score : scores, dashboard, export CSV
- feature/greenIT : UI, CSS sobre, page d'accueil
- feature/db : schéma BDD, migrations, déploiement

7.2 Conclusion

EvalConnect démontre qu'il est possible de créer une application web complète, sécurisée et fonctionnelle tout en respectant les principes du numérique durable. Avec des pages pesant en moyenne 4-5 Ko, 2-3 requêtes HTTP par page, et des scores PageSpeed de 100/100, le projet atteint ou dépasse tous les objectifs Green IT fixés dans le cahier des charges.

Section 8 — Annexes

Annexe A — Récapitulatif des livrables

#	Livrable	Emplacement	Statut
1	Code source front-end	GitHub /backend/templates + /backend/static	✓ Livré
2	Code source back-end	GitHub /backend/app.py	✓ Livré
3	Scripts base de données	GitHub /database/schema.sql + seed.sql	✓ Livré
4	README.md	GitHub (racine)	✓ Livré
5	Site déployé en ligne	https://numerique-durable.onrender.com	✓ Fonctionnel
6	Rapport PDF	GitHub /docs	✓ Ce document
7	Diagrammes UML	GitHub /docs + Livrable 1	✓ Livré
8	Wireframes	GitHub /docs + Livrable 1	✓ Livré
9	Captures EcoIndex / PageSpeed	Rapport Section 5 + Annexes	✓ Livré
10	Tableau de tests fonctionnels	Rapport Section 6	✓ Livré

Annexe B — Stack technique résumée

Composant	Technologie	Version
Langage back-end	Python	3.14
Framework web	Flask	3.1.0
Hashage MDP	bcrypt	4.1.3
BDD développement	SQLite	intégré Python
BDD production	PostgreSQL	Render Free Tier
Connecteur PostgreSQL	psycopg2-binary	2.9.12
Serveur production	Gunicorn	21.2.0
Hébergement	Render.com	Free Tier
Front-end	HTML5 / CSS3 / Vanilla JS	Natif